MAD 1 WEEK 2

Markup and HTML5 Basics

Information Representation

- **Computers work with bits:** Binary digits (0 and 1).
    - **Numbers:**
        - Binary numbers: Place value, e.g., 6 = 0110.
        - Two's complement: For negative numbers, e.g., -6 = 1010.
    - **Text Representation:** ASCII, Unicode, UTF-8.

ASCII (Can be 7 bit or 8 bit)

- **American Standard Code for Information Interchange.**
- 7-bit encoding: Represents 128 characters.
    - Includes letters (a-z, A-Z), numbers (0-9), and special characters (!@#$%).
    - Limitation: Insufficient for global characters like ㅋ or 不.

Unicode

- Supports global scripts and characters
    - UCS-2(UTF-16): 2 bytes =16 bits (65,536 characters).
    - UCS-4(UTF-32): 4 bytes =32 bits (4 billion+ characters).
    - UCS-1(UTF-8):1 bytes =8 bits

UTF-8

- Encodes characters efficiently:
    - Uses 8 bits for ASCII-compatible characters.
    - Expands for others with prefix codes.
    - Most common encoding used today.

Markup

Types of Markup

1. **Presentational:** WYSIWYG (What You See Is What You Get).
    - Focus on appearance.
    - Example: MS Word, Google Docs.

2. **Procedural:** Instructions for display.
    - Example: “Make text bold, indent 4 spaces.”
3. **Descriptive:** Focus on meaning and structure.
    - Example: HTML tags like `<title>` and `<heading>`.

HTML Basics

- **HTML:** HyperText Markup Language.
- Invented by Tim Berners-Lee (~1989).
- HTML is browser-interpreted and forgiving with loose syntax.

HTML Tags

- **Basic Syntax:**
    - Tags enclosed in angle brackets `<  >`.
    - Paired tags have opening and closing (`<p>Text</p>`).
    - Nesting: Tags must be properly nested (`<em><strong>Text</strong></em>`).
    - Difference between <p>,,
    - Cover parts of an url
    - Cover hexadecimal of ip address

HTML Example

```
<!DOCTYPE html>
<html>
<body>
  <h1>My First Heading</h1>
  <p>My first paragraph.</p>
</body>
</html>
```

HTML5 Features

- New logical elements: `<nav>`, `<footer>`, `<article>`.
- Media elements: `<audio>`, `<video>`.
- Removed deprecated tags like `<center>`, `<font>`.

Styling with CSS

- **CSS:** Cascading Style Sheets separate content and presentation.
    - Inline CSS: Styling directly inside tags.
      `<h1 style="color:blue;text-align:center;">Hello</h1>`

Internal CSS: Inside `<style>` in `<head>`.

```
<style>
    body { background-color: linen; }
    h1 { color: maroon; margin-left: 40px; }
  </style>
```

    - External CSS: Separate file linked to HTML.
      `<link rel="stylesheet" href="styles.css">`

CSS (Cascading Style Sheets)

CSS is a powerful language used for styling and visually enhancing HTML documents. With CSS, you can control the layout, color, fonts, and many other aspects of your web pages. There are multiple ways to apply CSS styles to HTML elements, giving you flexibility and control over the appearance of your website.

1. Inline CSS

- Inline CSS is the most basic and straightforward way to apply styles directly to individual HTML elements.
- By using the `<style>` attribute within an HTML tag, you can specify CSS properties and values.

Example:

```
<p style="color: orange; font-size: 24px;">

    This is a paragraph with inline styles applied.

</p>
```

Rendered HTML: This is a paragraph with inline styles applied.

2. Internal CSS

- Internal CSS allows you to define styles within the HTML document itself, typically within the `<head>` section.

- To apply internal CSS, you enclose the styles within the `<style>` tags.

Example:

```
<head>

    <style>

      p {

        background-color: blue;

        color: white;

        font-size: 16px;

      }

    </style>

  </head>

  <body>

    <p>This is a paragraph with internal styles applied.</p>

  </body>
```

Rendered HTML: This is a paragraph with internal styles applied.

3. External CSS

- External CSS is the recommended and most scalable approach for styling HTML documents.
- With this method, you create a separate CSS file with the .css extension and link it to your HTML document using the `<link>` tag.

Example: HTML file:

```
<head>

    <link rel="stylesheet" href="styles.css">
```

```
  </head>

  <body>

    <p>This is a paragraph with external styles applied.</p>

  </body>
```

CSS file (styles.css):

```
p {

    background-color: #c678dd;

    color: #282c34;

    font-size: 28px;

    font-weight: bold;

    font-style: italic;

    text-decoration: line-through;

  }
```

Rendered HTML: This is a paragraph with external styles applied.

4.CSS Selectors

Element Selector: Targets elements based on their tag name.

```
   p {

 color: blue;

  }
```

5.Class Selector: Targets elements with a specific class attribute.

```
<p class="highlight">This is a paragraph with class selector applied.</p>

.highlight {

    background-color: yellow;

  }
```

Rendered HTML: This is a paragraph with class selector applied.

6.ID Selector: Targets a specific element with a unique ID attribute.

```
<p id="top">Some para...</p>

#top {

    color: red;

    font-size: 24px;

  }
```

Rendered HTML: Some para…

7.Universal Selector: Targets all elements on the page.

```
*{

   color: cyan;

  }
```

Rendered HTML: This is a paragraph with universal selector applied.

Priority of CSS Selectors

1. **Inline styles** (highest priority)
2. **ID selectors**
3. **Class selectors**
4. **Tag selectors** (lowest priority)
5. `!important`: Overrides all others.

Responsive Design

- Ensures compatibility across devices (mobile, tablets, desktops).
- Frameworks like **Bootstrap** simplify responsive layouts.

Document Object Model (DOM)

- HTML document structure represented as a tree.

Example:

```
<html>
<head>
  <title>My Title</title>
</head>
<body>
  <h1>A Heading</h1>
  <a href="link">Link Text</a>
</body>
</html>
```

- **Manipulation:** APIs like JavaScript can directly modify the DOM.